학번: 202211414
이름: 김민재

[Assignment #2] Numerical Analysis 2

Due: October 5, 2026 (Monday), 11:59 PM

Submission: Upload to LMS (Two problems in total)

[Problem#1] (Textbook p.690의 11.1절 Example 2의 변형)

The BVP is given as

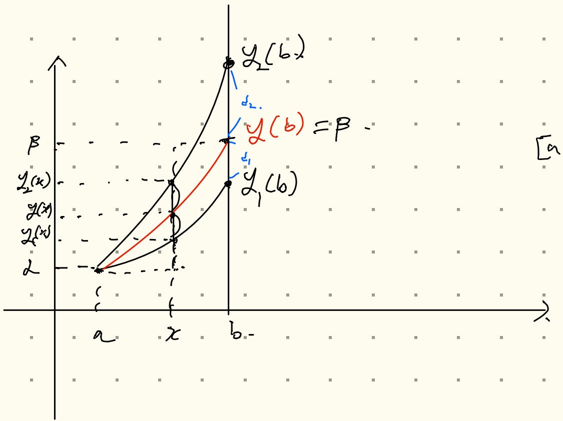
$$y'' = -\frac{2}{x}y' + \frac{2}{x^2}y + \frac{\sin(\ln x)}{x^2}, \quad 1 \le x \le 2, \quad y(1) = 1, \quad y(2) = 2.$$

The exact solution is $y(x) = c_1 x + \frac{c_2}{x^2} - \frac{3}{10}\sin(\ln x) - \frac{1}{10}\cos(\ln x)$,

where $c_2 = \frac{1}{70}[8 - 12\sin(\ln 2) - 4\cos(\ln 2)]$, $c_1 = \frac{11}{10} - c_2$.

For the BVP, complete the following tasks.

(1) Summarize the **linear shooting method** in your own words. Include its basic idea, the initial value problems it uses, and how their solutions are combined to satisfy the boundary conditions. Write your explanation by hand.



가정.| 선형인 미분 방정식

$y'' = py' + qy + r$ 에 대하여

[a,b] 에서의 초기 기울기 t 에 대하여 y(x) 는 비례하며.
↳ linear

초기 기울기가 각각 t_1, t_2, t_3 인

y_1, y_2, y_3 의 세 선형 미분 방정식의.

$\frac{y_2 - y_1}{y_3 - y_1} = \frac{t_2 - t_1}{t_3 - t_1}$ 로 초기 기울기의 비와 [a,b] 구간에서의

세 방정식 y의 비는 동일하다.

↝ 증명과정은 2page 에 있습니다.

⇒

위 가정을 통하여 x=b, x=x 에서의 비가 동일함을 가지고. 비례식 구해 해 y(x)를 구할 수 있다.

$$y(x) - y_1(x) : y(x) - y_2(x) = \beta - y_1(b) : y_2(b) - \beta$$

$$(y(x) - y_2(x))(\beta - y_1(b)) = (y(x) - y_1(x))(y_2(b) - \beta)$$

$$y(x)\left((\beta - y_1(b)) - (\beta - y_2(b))\right) = y_2(x)(\beta - y_1(b)) - y_1(x)(\beta - y_2(b))$$

$$\Rightarrow y(x)\left(y_2(b) - y_1(b)\right) = y_2(x)(\beta - y_1(b)) - y_1(x)(\beta - y_2(b))$$

$$y(x) = \left(\frac{\beta - y_1(b)}{y_2(b) - y_1(b)}\right) y_2(x) - \left(\frac{\beta - y_2(b)}{y_2(b) - y_1(b)}\right) y_1(x).$$

BVP, 선형 슈팅문제 풀이 단계.

step 1.

기울기 t_1, t_2 에 대하여 IVP 해결(오일러 방법, RK4) 을 통해. a, ... b 까지의 $\frac{b-a}{h}$개의 구간 해를 구한다.

step2.

$$y(x) = \left(\frac{\beta - y_1(b)}{y_2(b) - y_1(b)}\right) y_2(x) - \left(\frac{\beta - y_2(b)}{y_2(b) - y_1(b)}\right) y_1(x).$$

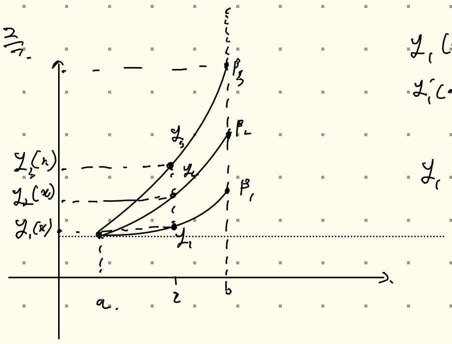
다음의 해 방정식을 통하여

$x_i = a + ih$ 에 대하여 b까지 반복하며 결과를 얻어 y(x)의 a, b까지의 해를 구해 낼 수 있다.

가정의 증명.

$[a, b]$ 영역에 대하여 $x=a$ 의 [illegible]
초기 기울기 $y'(a)=t$ 에 대하여 $y(x)$가 비례하도록 하는 $y''=py'+qy+r(x)$ 인 선형 방정식 y_1, y_2 에 대해.

즉.



$y_1(a)=\alpha,\ y_2(a)=\alpha,\ y_3(a)=\alpha$

$y_1'(a)=t_1,\ y_2'(a)=t_2,\ y_3'(a)=t_3.$

y_1, y_2, y_3 모두 동일한 $p(x), q(x)$ 의
$y''=py'+qy+r$ 일 때.

미분방정식

$L[y]=a_0(x)y''+a_1(x)y'+a_2(x)y+r(x)=0$

와 같이 y'', y', y 가 따로 이루어진 경우 선형이며

이 경우

$L[y_1+y_2]=L[y_1]+L[y_2]$

상수 c 에 대해

$L[cy]=cL[y]$ 가 보장된다.

이를 통해 $y''=py'+qy+r$ 또한

$L[y]=-y''+py'+qr+r=0$ 임으로

선형성을 가짐을 알 수 있으며

이때

y_1, y_2, y_3 또한 모두 선형임을 알 수 있다.

이 경우

② $y_2(x)=c_1y_1(x)+c_2y_3(x)$, $c_1+c_2=1$

의 선형 결합을 통해

c_1 과 c_2 를 구해내는 과정을 통해

① 식을 증명한다.

⇒ 선형성을 가진

y_1, y_3 에 대하여 다음과 같이 나타낼 수 있다.

$L[y_1]=-y_1''+py_1'+qy_1+r,$

$L[y_3]=-y_3''+py_3'+qy_3+r,$

$L[y]=c_1L[y_1]+c_2L[y_3],\ (c_1+c_2=1)$

② 의 선형결합을 통해 y_2를 만들기 위해.

$y_2(x)=c_1y_1(x)+(1-c_1)y_3(x)\ \cdots\cdots 1)$

다음을 선형결합할 수 있으며 이를 미분하여

$y_2'(x)=c_1y_1'(x)+(1-c_1)y_3'(x)\ \cdots\cdots 2)$

다음을 얻을 수 있다

③ 에 의해서 2계 선형 미분방정식은 독립변수(y', y)에 의해 결정되며 $y_1', y_1,\ y_3', y_3,\ y_2', y_2$ 을 통해 c_1, c_2를 구해낼 수 있다.

$$\left.\begin{matrix} x=a, & y_1'=t_1 & y_2'=t_2 & y'=t_3 \\ x=b & y_1=\beta_1 & y_2=\beta_2 & y_3=\beta_3 \end{matrix}\right) \Rightarrow \begin{cases} \beta_2=c_1\beta_1+(1-c_1)\beta_3 \\ t_2=c_1t_1+(1-c_1)t_3 \end{cases}$$

다음과 같이 연립방정식으로 표현된다.

$\Rightarrow\ \beta_2=c_1(\beta_1-\beta_3)+\beta_3$

$t_2=c_1(t_1-t_3)+t_3$

$$c_1=\frac{\beta_2-\beta_3}{(\beta_1-\beta_3)} \quad 1-c_1=\frac{-\beta_2+\beta_1}{(\beta_1-\beta_3)}$$

$$c_1=\frac{t_2-t_3}{t_1-t_3} \quad 1-c_1=\frac{-t_2+t_1}{t_1-t_3}$$

으로.

$$y_2(x)=\frac{\beta_2-\beta_3}{(\beta_1-\beta_3)}y_1(x)-\frac{\beta_2-\beta_1}{(\beta_1-\beta_3)}y_3(x)$$ 의 해를 얻을 수 있으며.

$$\frac{\beta_2-\beta_3}{\beta_1-\beta_3}=\frac{t_2-t_3}{t_1-t_3}$$ ⇒ 기울기 변화의 비가 $\underset{[a,b]}{y(x)}$ 의 변화 비와 동일하다는 가정을 증명할 수 있다.

+③ 2계도 미분방정식의 선형성과 해의 결정 과정. (with Copilot)

$L[y_2]=a_0(x)y_2''+a_1(x)y_2'+a_2(x)y_2+r(x)$, 이때.

에 대하여

$L[y_1], L[y_2]$ 의 경우

$L[y_1+y_2]=L[y_1]+L[y_2]$

$c_1L[y_1]=L[c_1y_1]$

$c_2L[y_2]=L[c_2y_2]$

$L[c_1y_1+c_2y_2]=c_1L[y_1]+c_2L[y_2]$

다음과 같이 상수배의 합에 대한 선형성이 보장되며 (비동차의 경우 $c_1+c_2=1$)

임의의 2계도 함수까지 미분가능한 $y(x)=c_1y_1(x)+c_2y_2(x)$ 를 표현할 수 있다.

$y''(x)=a_0(x)y'(x)+a_1(x)y(x)$ 의 해 $y(x)$를 구하기 위해서는 $\underline{y'(0), y(0)}$ 의 값이 필요하다.

↳ 꼭 이들이 아니더라도.
$y''(0), y'(0)$ …… 등등
$y''(\alpha), y'(\beta)$ 서로 독립인 변수이면 가능. 2개면 (2차원).
but. BVP 문제에서 $y'(0), y(0)$를 계산.

즉 앞서 구한 임의의 해 $y(x)=c_1y_1(x)+c_2y_2$는 $y(0), y'(0)$의 값으로 결정 가능. 이때.

$y(x)=c_1y_1(x)+c_2y_2(x)$. 는 도함수 합 법칙에 따라.

$y'(x)=c_1y_1'(x)+c_2y_2'(x)$ 도 가능하다.

이때 $y(0), y_1(0), y_2(0)$, 가 주어진다면.
$y'(0), y_1'(0), y_2'(0)$

연립방정식으로 아래와 같이 c_1, c_2를 나타낼 수 있다.

$$\begin{bmatrix} y(0) \\ y'(0) \end{bmatrix}=\begin{bmatrix} y_1(0)+y_2(0) \\ y_1'(0)+y_2'(0) \end{bmatrix}\begin{bmatrix} c_1 \\ c_2 \end{bmatrix}$$

즉 $y(0), y_1(0), y_2(0)$ / $y'(0), y_1'(0), y_2'(0)$ 해당 조건으로 c_1, c_2를 구할 수 있다.

(2) Implement the linear shooting method using the **Euler's method** in Python with $N = 10$. Plot the approximations and given exact solutions on the same graph.

(3) Implement the linear shooting method using the **RK4 method (Algorithm 11.1)** in Python with $N = 10$. Plot the approximations and given exact solutions on the same graph.

(4) Print and compare the errors of the approximations from (2) and (3) against the exact solution. Since the errors may be too similar to distinguish visually in the plots, print their numerical values and compare them.

(5) For each method in (2) and (3), compare the errors obtained with $N = 10$ and $N = 100$.

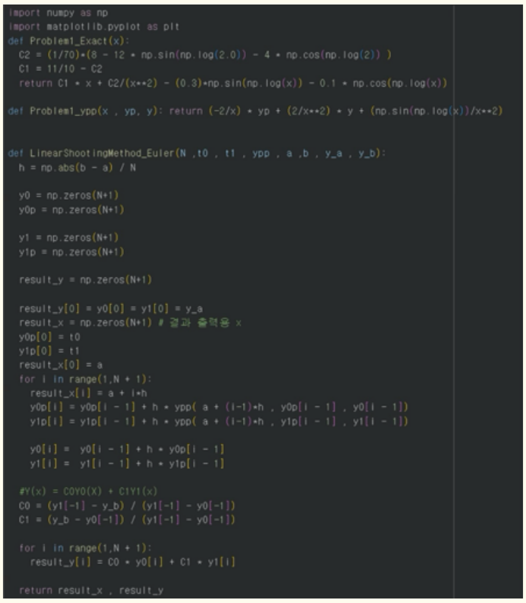
[Problem#1] (Textbook p.690의 11.1절 Example 2의 변형)

모든 문제 $t_1 = 0$ $t_2 = 1$로 정의.
초기 기울기

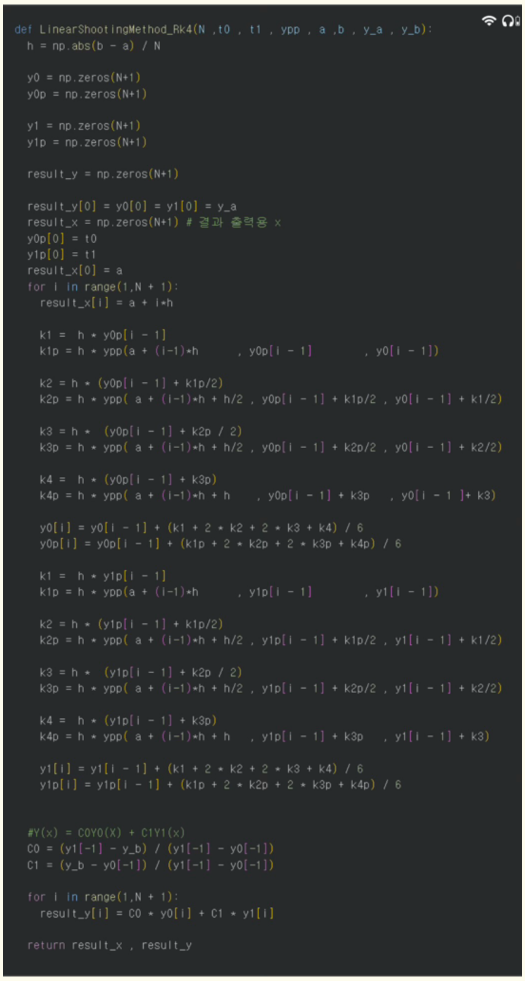
The BVP is given as

$$y'' = -\frac{2}{x}y' + \frac{2}{x^2}y + \frac{\sin(\ln x)}{x^2},\ \ 1 \le x \le 2,\ \ y(1) = 1,\ \ y(2) = 2.$$

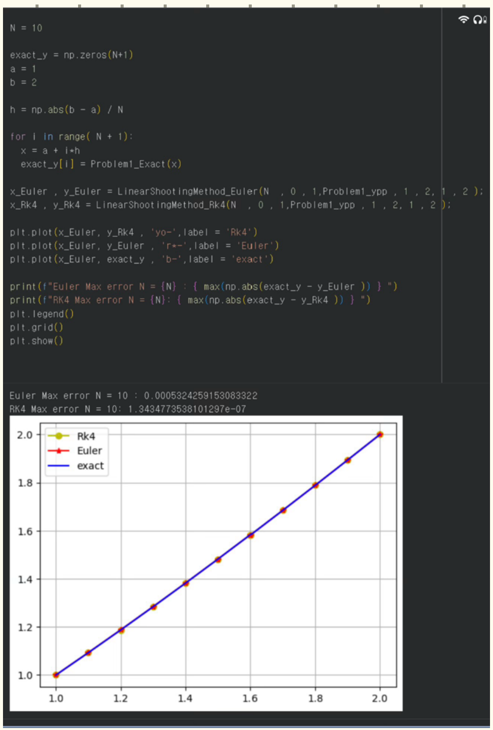
The exact solution is $y(x) = c_1 x + \frac{c_2}{x^2} - \frac{3}{10}\sin(\ln x) - \frac{1}{10}\cos(\ln x)$,
where $c_2 = \frac{1}{70}[8 - 12\sin(\ln 2) - 4\cos(\ln 2)]$, $c_1 = \frac{11}{10} - c_2$.



```
import numpy as np
import matplotlib.pyplot as plt
def Problem1_Exact(x):
  C2 = (1/70)*(8 - 12 * np.sin(np.log(2.0)) - 4 * np.cos(np.log(2)) )
  C1 = 11/10 - C2
  return C1 * x + C2/(x**2) - (0.3)*np.sin(np.log(x)) - 0.1 * np.cos(np.log(x))

def Problem1_ypp(x , yp, y): return (-2/x) * yp + (2/x**2) * y + (np.sin(np.log(x))/x**2)


def LinearShootingMethod_Euler(N ,t0 , t1 , ypp , a ,b , y_a , y_b):
  h = np.abs(b - a) / N

  y0 = np.zeros(N+1)
  y0p = np.zeros(N+1)

  y1 = np.zeros(N+1)
  y1p = np.zeros(N+1)

  result_y = np.zeros(N+1)

  result_y[0] = y0[0] = y1[0] = y_a
  result_x = np.zeros(N+1) # 결과 출력용 x
  y0p[0] = t0
  y1p[0] = t1
  result_x[0] = a
  for i in range(1,N + 1):
    result_x[i] = a + i*h
    y0p[i] = y0p[i - 1] + h * ypp( a + (i-1)*h , y0p[i - 1] , y0[i - 1])
    y1p[i] = y1p[i - 1] + h * ypp( a + (i-1)*h , y1p[i - 1] , y1[i - 1])

    y0[i] =  y0[i - 1] + h * y0p[i - 1]
    y1[i] =  y1[i - 1] + h * y1p[i - 1]

  #Y(x) = C0Y0(X) + C1Y1(x)
  C0 = (y1[-1] - y_b) / (y1[-1] - y0[-1])
  C1 = (y_b - y0[-1]) / (y1[-1] - y0[-1])

  for i in range(1,N + 1):
    result_y[i] = C0 * y0[i] + C1 * y1[i]

  return result_x , result_y
```

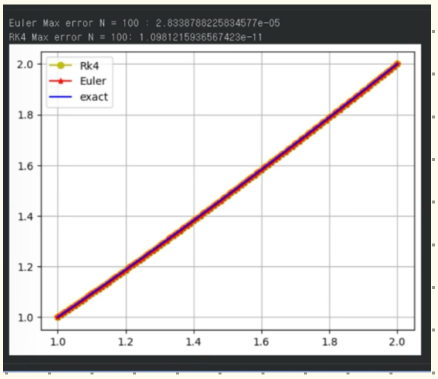


```
def LinearShootingMethod_Rk4(N ,t0 , t1 , ypp , a ,b , y_a , y_b):
  h = np.abs(b - a) / N

  y0 = np.zeros(N+1)
  y0p = np.zeros(N+1)

  y1 = np.zeros(N+1)
  y1p = np.zeros(N+1)

  result_y = np.zeros(N+1)

  result_y[0] = y0[0] = y1[0] = y_a
  result_x = np.zeros(N+1) # 결과 출력용 x
  y0p[0] = t0
  y1p[0] = t1
  result_x[0] = a
  for i in range(1,N + 1):
    result_x[i] = a + i*h

    k1 =  h * y0p[i - 1]
    k1p = h * ypp(a + (i-1)*h      , y0p[i - 1]        , y0[i - 1])

    k2 = h * (y0p[i - 1] + k1p/2)
    k2p = h * ypp( a + (i-1)*h + h/2 , y0p[i - 1] + k1p/2 , y0[i - 1] + k1/2)

    k3 = h *  (y0p[i - 1] + k2p / 2)
    k3p = h * ypp( a + (i-1)*h + h/2 , y0p[i - 1] + k2p/2 , y0[i - 1] + k2/2)

    k4 =  h * (y0p[i - 1] + k3p)
    k4p = h * ypp( a + (i-1)*h + h    , y0p[i - 1] + k3p   , y0[i - 1 ]+ k3)

    y0[i] = y0[i - 1] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
    y0p[i] = y0p[i - 1] + (k1p + 2 * k2p + 2 * k3p + k4p) / 6

    k1 =  h * y1p[i - 1]
    k1p = h * ypp(a + (i-1)*h      , y1p[i - 1]        , y1[i - 1])

    k2 = h * (y1p[i - 1] + k1p/2)
    k2p = h * ypp( a + (i-1)*h + h/2 , y1p[i - 1] + k1p/2 , y1[i - 1] + k1/2)

    k3 = h *  (y1p[i - 1] + k2p / 2)
    k3p = h * ypp( a + (i-1)*h + h/2 , y1p[i - 1] + k2p/2 , y1[i - 1] + k2/2)

    k4 =  h * (y1p[i - 1] + k3p)
    k4p = h * ypp( a + (i-1)*h + h   , y1p[i - 1] + k3p   , y1[i - 1] + k3)

    y1[i] = y1[i - 1] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
    y1p[i] = y1p[i - 1] + (k1p + 2 * k2p + 2 * k3p + k4p) / 6


  #Y(x) = C0Y0(X) + C1Y1(x)
  C0 = (y1[-1] - y_b) / (y1[-1] - y0[-1])
  C1 = (y_b - y0[-1]) / (y1[-1] - y0[-1])

  for i in range(1,N + 1):
    result_y[i] = C0 * y0[i] + C1 * y1[i]

  return result_x , result_y
```



```
N = 10

exact_y = np.zeros(N+1)
a = 1
b = 2

h = np.abs(b - a) / N

for i in range( N + 1):
  x = a + i*h
  exact_y[i] = Problem1_Exact(x)

x_Euler , y_Euler = LinearShootingMethod_Euler(N  , 0 , 1,Problem1_ypp , 1 , 2, 1 , 2 );
x_Rk4 , y_Rk4 = LinearShootingMethod_Rk4(N  , 0 , 1,Problem1_ypp , 1 , 2, 1 , 2 );

plt.plot(x_Euler, y_Rk4 , 'yo-',label = 'Rk4')
plt.plot(x_Euler, y_Euler , 'r*-',label = 'Euler')
plt.plot(x_Euler, exact_y , 'b-',label = 'exact')

print(f"Euler Max error N = {N} : { max(np.abs(exact_y - y_Euler )) } ")
print(f"RK4 Max error N = {N}: { max(np.abs(exact_y - y_Rk4 )) } ")
plt.legend()
plt.grid()
plt.show()
```

```
Euler Max error N = 10 : 0.0005324259153083322
RK4 Max error N = 10: 1.3434773538101297e-07
```

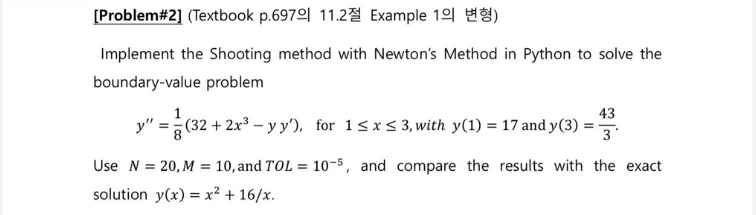


```
Euler Max error N = 100 : 2.8338788225834577e-05
RK4 Max error N = 100: 1.0981215936567423e-11
```

- 수행 결과

  N = 10 에서부터 Rk4 와 Euler 방법 모두
  매우 실제 해와 매우 근사되어지며
  에러의 경우 Rk4의 에러가 Euler 방법에 비하여 매우 작게 나옴을
  알 수 있다.

  N = 100으로 h 즉 step size를 줄인 결과
  Euler, Rk4 모두 에러가 크게 감소하였으며
  여전히 Rk4의 에러가 작게 나옴을 알 수 있다.



[Problem#2] (Textbook p.697의 11.2절 Example 1의 변형)

Implement the Shooting method with Newton's Method in Python to solve the boundary-value problem

$$y'' = \frac{1}{8}(32 + 2x^3 - y\,y'),\quad \text{for } 1 \le x \le 3, \text{with } y(1) = 17 \text{ and } y(3) = \frac{43}{3}.$$

Use $N = 20, M = 10$, and $TOL = 10^{-5}$, and compare the results with the exact solution $y(x) = x^2 + 16/x$.

$y'' = 4 + \frac{1}{4}x^3 - \underline{y \cdot y'}$ 의 경우 $[a,b]$ 에서의

$y \cdot y'$ 다음의 항이 1차식이 아니므로 비선형 방정식이다

⇒ 선형관계를 통한 정확한 초기 기울기를 한번에 찾을수 없다.

⇒ 반복법을 통하여 초기 기울기 t에 대하여 그 결과가 $y(b)$ 에 근사시키는 t를 찾아야 한다.

t에 대하여 $x : a \to b$로 IVP를 통하여 찾아낸 값을.

$y(b,t)$라 한다면

실제 값 $y(b) = \beta$ 에 대한 오차를

$F(t) = y(b,t) - \beta$ 의 방정식 형태로 나타낼 수 있다.

이때. $F(t) = 0$ 에 근사 시키는 t를 찾는 과정을 통해

적절한 초기값 t를 구할 수 있다.

⇒ $F(t) = y(b,t) - \beta = 0$ 의 해 t를 찾기

위한 뉴턴 방법은 다음과 같다.

$$t_k = t_{k-1} - \frac{F(t{-}1)}{F'(t{-}1)} = t_{k-1} - \frac{y(b,t_{k-1}) - \beta}{\frac{\partial y}{\partial t}(b,t_{k-1})}$$

이때. $F(t_k) < TOL$ 이 종료 조건이 된다.

$\frac{\partial y}{\partial t}(b,t)$ 를 구하기 위해서

$\frac{\partial}{\partial t} \cdot y''$ 에 대해서 IVP를 통해 구해야 한다.

이때.

$$\frac{\partial}{\partial t} f(x,y,y') = \frac{\partial f}{\partial x} \cdot \underset{\substack{\downarrow \\ x\text{와 } t(0) \\ \text{독립변수.}}}{\frac{\partial x}{\partial t}} + \frac{\partial f}{\partial y}\frac{\partial y}{\partial t} + \frac{\partial f}{\partial y'} \cdot \frac{\partial y'}{\partial t}$$

$$= \frac{\partial f}{\partial y}\frac{\partial y}{\partial t} + \frac{\partial f}{\partial y'} \cdot \frac{\partial y'}{\partial t}$$ 이때. $\frac{\partial y}{\partial t}$를 $z(x)$로 정의.

$$z'' = \frac{\partial f}{\partial y} z + \frac{\partial f}{\partial y'} z'$$ (로 둘 수 있다.)

z, IVP를 위해 $z(a)$와 $z'(a)$의 초기값이 필요.

$z(a)$는 $y(a)$에 t에 대해 미분. $y(a) = \alpha \to$ 상수.

$z(a) = 0$

$z'(a)$는 $y'(a)$에 t에 대해 미분 $y'(a) = t$.

$z'(a) = 1$.

z에 대한 IVP를 구할 수 있다.

$N = 20$ ⇝ 구간 $M = 10$ ~ max.

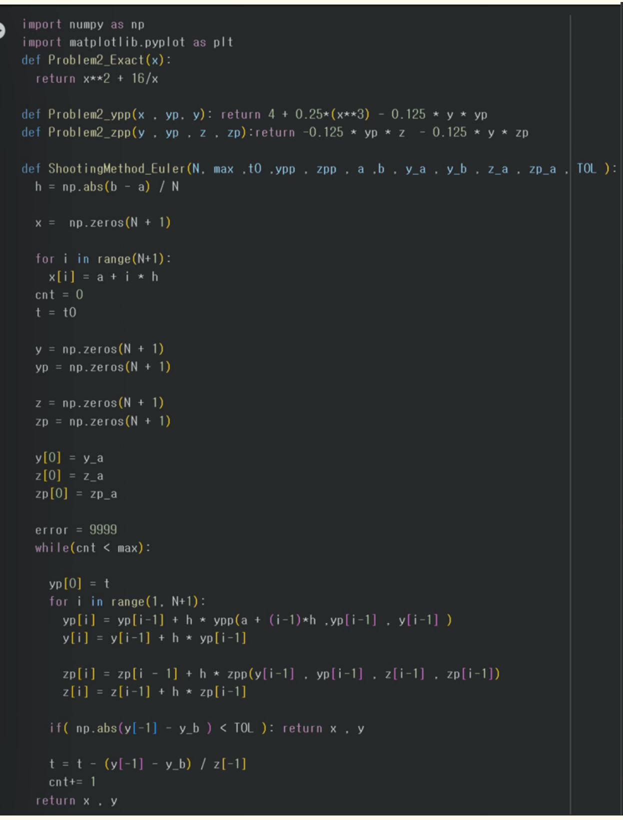
$TOL = 10^{-5}$

Step. 1. z에 대한 식도출.

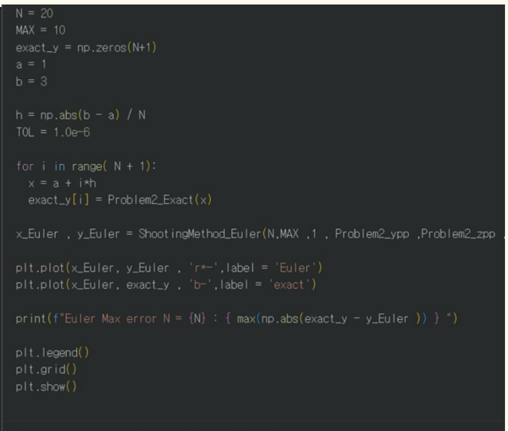
$z'' = \frac{\partial f}{\partial y} z + \frac{\partial f}{\partial y'} z'$ ⇒ $z'' = -\frac{1}{8}y' \cdot z - \frac{1}{8} y z$. $z(a) = 0$, $z'(a) = 1$

$f(x,y,y') = 4 + \frac{1}{4}x^3 - \frac{1}{8}y \cdot y'$ ↑ $y'' = 4 + \frac{1}{4}x^3 - \frac{1}{8}y \cdot y'$ $y(a) = \alpha$, $y'(a) = t$. $t_0 = 0$.

$\left(\frac{\partial f}{\partial y} = -\frac{1}{8}y' \quad \frac{\partial f}{\partial y'} = -\frac{1}{8}y\right)$ $y(b) = \beta$ $\alpha = 17$ $\beta = \frac{43}{3}$ $a = 1$, $b = 3$



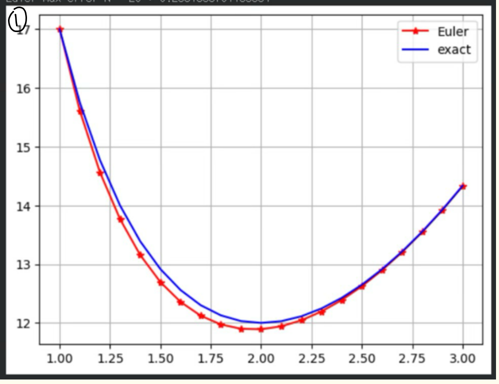
```python
import numpy as np
import matplotlib.pyplot as plt
def Problem2_Exact(x):
  return x**2 + 16/x

def Problem2_ypp(x , yp, y): return 4 + 0.25*(x**3) - 0.125 * y * yp
def Problem2_zpp(y , yp , z , zp):return -0.125 * yp * z  - 0.125 * y * zp

def ShootingMethod_Euler(N, max ,t0 ,ypp , zpp , a ,b , y_a , y_b , z_a , zp_a , TOL ):
  h = np.abs(b - a) / N

  x =  np.zeros(N + 1)

  for i in range(N+1):
    x[i] = a + i * h
  cnt = 0
  t = t0

  y = np.zeros(N + 1)
  yp = np.zeros(N + 1)

  z = np.zeros(N + 1)
  zp = np.zeros(N + 1)

  y[0] = y_a
  z[0] = z_a
  zp[0] = zp_a

  error = 9999
  while(cnt < max):

    yp[0] = t
    for i in range(1, N+1):
      yp[i] = yp[i-1] + h * ypp(a + (i-1)*h ,yp[i-1] , y[i-1] )
      y[i] = y[i-1] + h * yp[i-1]

      zp[i] = zp[i - 1] + h * zpp(y[i-1] , yp[i-1] , z[i-1] , zp[i-1])
      z[i] = z[i-1] + h * zp[i-1]

    if( np.abs(y[-1] - y_b ) < TOL ): return x , y

    t = t - (y[-1] - y_b) / z[-1]
    cnt+= 1
  return x , y
```



```python
N = 20
MAX = 10
exact_y = np.zeros(N+1)
a = 1
b = 3

h = np.abs(b - a) / N
TOL = 1.0e-6

for i in range( N + 1):
  x = a + i*h
  exact_y[i] = Problem2_Exact(x)

x_Euler , y_Euler = ShootingMethod_Euler(N,MAX ,1 , Problem2_ypp ,Problem2_zpp

plt.plot(x_Euler, y_Euler , 'r*-',label = 'Euler')
plt.plot(x_Euler, exact_y , 'b-',label = 'exact')

print(f"Euler Max error N = {N} : { max(np.abs(exact_y - y_Euler )) } ")

plt.legend()
plt.grid()
plt.show()
```

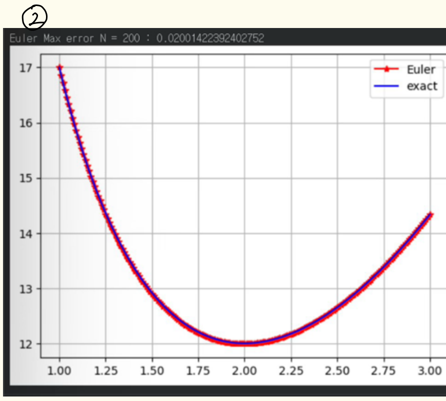
```
Euler Max error N = 20 : 0.2331588701188334
```

①



②

```
Euler Max error N = 200 : 0.02001422392402752
```



수행 결과

다음과 같이 BVP 조건의 비선형 방정식을 근사시킬 수 있었다. ① $N=20$, $M=10$ 조건에서 전반적인 개형은 근사하나 그 값의 오차가 크게 보이는 모습을 볼 수 있다.

이를 위해

② 와 같이 N을 키워 step을 늘리거나 IVP의 고차 근사가 필요해 보여진다.